

Subtask 3 - Decoder-Only Transformers for Poetry Generation

Overview

In this task, you will explore autoregressive sequence modeling through poetry generation using decoder-only transformers (GPT-style architectures).

The focus of the subtask is on:

- understanding how different sequence models generate text,
 - experimenting with architectural design choices,
 - studying the effect of attention in sequence modeling,
 - and analyzing the strengths and limitations of different approaches.
-

Dataset

We will use a poetry corpus for this assignment:

[Poetry Dataset](#)

Suggested Workflow

The following is only a rough guideline. You are strongly encouraged to explore beyond this.

1. Build Baselines

Start with simpler autoregressive language models before moving to transformers.

Possible baselines include:

- Bigram models

- MLP-based language models
- RNNs
- LSTMs

Establish baseline generation quality and compare:

- coherence,
- training behavior,
- stylistic consistency,
- and limitations of sequential architectures.

2. Decoder-Only Transformer

Implement and train a GPT-style decoder-only transformer from scratch.

This includes:

- causal self-attention,
- positional encodings,
- masking mechanisms,
- transformer decoder blocks,
- autoregressive training/inference.

You may use PyTorch/TensorFlow primitives, but transformer blocks and attention mechanisms should be implemented by you and not imported from high-level libraries.

The purpose is to understand the architecture deeply rather than simply using pretrained APIs.

3. Inference & Decoding Experiments

Experiment with different decoding strategies during inference.

Possible directions:

- greedy decoding
- temperature sampling
- top-k sampling
- top-p sampling
- repetition penalties
- prompt conditioning

Analyze how decoding strategies affect:

- creativity,
- repetition,

- coherence,
 - and stylistic quality.
-

Ablations and Experiments

An optional component of this assignment is experimentation and analysis.

You are expected to conduct meaningful ablation studies and architectural comparisons.

Possible axes of experimentation include:

Architecture

- Bigram vs MLP vs RNN vs LSTM vs Transformer
- shallow vs deep transformers
- attention vs no-attention models
- adding attention mechanisms to recurrent architectures
- decoder depth
- residual connections
- normalization variants

Tokenization

- character-level tokenization
- word-level tokenization
- subword/BPE tokenization
- vocabulary size effects

Training

- learning rate
- batch size
- optimizer choice
- dropout
- gradient clipping
- context length
- training stability

Decoding

- greedy decoding
- top-k sampling
- top-p sampling
- temperature effects
- repetition penalties

Representation & Attention

- positional encoding variants
- attention visualization
- head specialization
- long-range dependency behavior

Data

- training on different poetry styles
- robustness to longer prompts
- mixing poetry datasets/styles
- stylistic conditioning

Try to visualize and interpret your results rather than only reporting metrics.

Deliverables

1. Source Code

Submit:

- training scripts,
- model implementations,
- preprocessing code,
- evaluation/inference code.

Your repository/notebook should be clean and reproducible.

2. Experimental Report

A short report containing:

- methodology,
- architectural choices,

- experiments conducted,
- ablation studies,
- observations,
- evaluation metrics,
- qualitative outputs,
- and analysis of model behavior.

3. Trained Model + Demo

Submit:

- trained checkpoints,
- inference notebook with a demo, results and any plots,
- optional Streamlit/Gradio demo.

Suggested demo ideas:

- poetry generation interface,
 - side-by-side comparison of different models,
 - attention visualization,
 - decoding strategy comparison.
-

Suggested Metrics

You may use:

- validation loss
- perplexity
- token-level accuracy
- generation diversity
- repetition statistics
- qualitative evaluation of outputs